



SENG 31242 – System Design Project

High-Level System Architecture

Specification

Group 10

SE/2022/046 - Lochithya Hettiarachchi

SE/2022/005 - Anushka Dilinuwan

SE/2022/010 - Adhithya Jayawardhana

SE/2022/025 - Vindya Wickramathilaka

SE/2022/041 - Ramesh Krishan

Supervisor – Eng. Dr. Tiroshan Madushanka

23/05/2026



1. Introduction

SithCare is a web-based mental health care platform developed to improve accessibility, privacy, and efficiency of mental health services in Sri Lanka. The platform provides appointment booking, telemedicine consultations, emergency mental health assistance, anonymous community consultation, AI-assisted specialist recommendation, volunteer listener support, and clinic administration services.

The system is designed as a scalable and modular healthcare platform capable of supporting multiple stakeholders including patients, mental health specialists, clinic administrators, volunteers, moderators, and system administrators.

This document describes the architectural design, component structure, deployment strategy, scalability considerations, and system-level technical decisions adopted for the SithCare platform.

2. Architectural Pattern Selected

SithCare will employ a Client-Server Architecture utilizing a Layered (N-Tier) model on the backend, tightly integrated with the Model-View-Controller (MVC) design pattern. The application is built using the MERN stack (MongoDB, Express.js, React.js, Node.js).

The layers are defined as follows:

- **Presentation Layer (Client):** A React-based Single Page Application (SPA) handling UI, routing, and state management for Patients, Specialists, and Admins.
- **Application/Controller Layer (Server):** Express.js REST APIs that intercept HTTP requests, validate payloads, and manage Role-Based Access Control (RBAC) via JSON Web Tokens (JWT).
- **Business Logic Layer (Server):** Node.js services managing core functionalities such as appointment scheduling, queue management, AI triage (SithBot), and emergency routing.
- **Data Access Layer (Database):** Mongoose Object Data Modeling (ODM) wrapping a NoSQL MongoDB database.



High-Level System Architecture Specification – SithCare Channeling App

The architecture is designed to:

- Separate system responsibilities clearly,
 - Simplify maintenance and debugging,
 - Improve scalability,
 - Support secure healthcare workflows,
 - Enable future migration toward microservice-based deployment.
-
- Instead of implementing a fully distributed microservices architecture during the academic project phase, the platform uses a modular service-oriented structure where each major functionality is isolated into independent service modules.
-
- This approach provides:
 - Enterprise-level scalability planning,
 - Easier future service decomposition,
 - Better maintainability,
 - Reduced development complexity.

3. Justification Referencing Quality Attributes

The chosen architecture directly supports the Non-Functional Requirements (NFRs) defined in the SRS:

3.1 Scalability (NFR-SC-01 to 03, NFR-P-03)

Node.js utilizes a non-blocking, event-driven I/O model which makes it exceptionally well-suited to handle the target metric of 500 concurrent users and 5x traffic spikes (NFR-SC-03). The stateless REST API design allows for horizontal scaling. MongoDB's flexible schema easily accommodates the schema-per-tenant design required for managing multiple clinic branches (NFR-SC-02).

3.2 Maintainability (NFR-M-01 to NFR-M-05)

The MVC and Layered approach enforces a strict separation of concerns. Frontend UI modifications will not impact backend business logic. This structure aligns perfectly with the development team's expertise and allows for parallel development. Using a single language (JavaScript) across the entire stack reduces context switching and improves code maintainability.



High-Level System Architecture Specification – SithCare Channeling App

3.3 Security & Privacy (NFR-S-01 to NFR-S-08)

A dedicated Controller layer acts as a gatekeeper, verifying JWTs and enforcing RBAC (Patient, Doctor, Admin) before any business logic is executed. MongoDB allows for field-level encryption, aiding in PDPA compliance (NFR-S-02), and the isolated backend layer prevents direct database exposure to the internet.

3.4 Reliability & Performance (NFR-R-01, NFR-P-01)

React's virtual DOM ensures fast UI rendering, meeting the sub-3-second page load time requirement (NFR-P-01). The separation of external services (PayHere, Jitsi) from the core backend logic ensures that failures in third-party integrations do not crash the primary application, satisfying the 99.5% uptime goal.

3.5 Usability & User Experience (NFR-U-01 to NFR-U-05)

The React-based frontend combined with Tailwind CSS provides a clean, responsive, and intuitive user experience. Simplified navigation flows, queue tracking, emergency shortcuts, and low-complexity interfaces reduce cognitive load for mental health patients and improve overall usability.

4. High-Level System Components

The SithCare platform consists of the following major components:

1. Patient Module

Handles:

- Registration and authentication
- Therapist searching
- Appointment booking
- Queue tracking
- Emergency support
- Community interaction



High-Level System Architecture Specification – SithCare Channeling App

2. Mental Health Specialist Module

Handles:

- Appointment management
- Schedule management
- Telemedicine consultations
- Patient communication

2. Clinic Administration Module

Handles:

- Clinic profile management
- Doctor allocation
- Queue management
- Appointment monitoring

4. SithCare Admin Module

Handles:

- Tracking of all the 3 portals
- Actions related to users, clinics, doctors
- Risk controlling and maintaining the progress of the application

5. Community Support Module

Handles:

- Anonymous consultation
- Recovery stories
- Peer support
- Volunteer listener sessions

6. AI Assistance Module (SithBot)

Handles:

- Symptom-based triage
- Specialist recommendations
- Navigation assistance



4. Component / Package Diagram Specifications

Below is the descriptive specification for the Component Diagram. (Please generate the visual diagram in Draw.io or UML tool of choice using these exact components):

- Frontend Components (React): AuthModule, Patient-Dashboard, Doctor-Portal, Clinic-AdminPanel, SithCare-AdminPanel, SithBotWidget, VideoConsultationRoom.
- Backend API Packages (Node/Express): AuthController, AppointmentService, QueueManager, NotificationService, PaymentGatewayAdapter (PayHere), EmergencyHandler.
- External Interfaces: PayHere Checkout API, Jitsi Meet SDK, Twilio SMS API, Nodemailer Email Service, Google Maps API

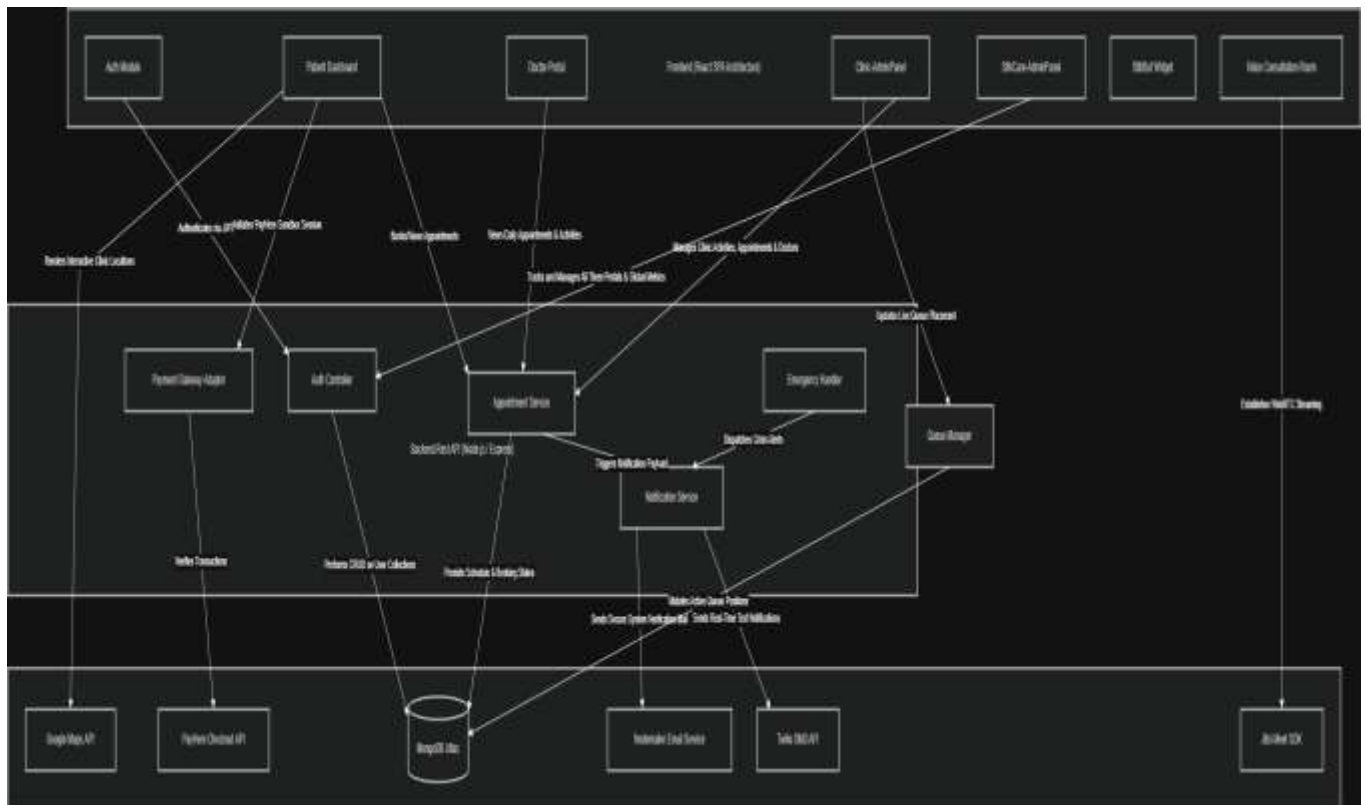


Figure 1 : Component Diagram



High-Level System Architecture Specification – SithCare Channeling App

Mermaid.js Diagram source code

flowchart TB

```
subgraph Frontend ["Frontend (React SPA Architecture)"]
```

```
  AM[Auth Module]
```

```
  PD[Patient Dashboard]
```

```
  DP[Doctor-Portal]
```

```
  CAP[Clinic-AdminPanel]
```

```
  SAP[SithCare-AdminPanel]
```

```
  SB[SithBot Widget]
```

```
  VC[Video Consultation Room]
```

```
end
```

```
subgraph Backend ["Backend Rest API (Node.js / Express)"]
```

```
  AC[Auth Controller]
```

```
  AS[Appointment Service]
```

```
  QM[Queue Manager]
```

```
  NS[Notification Service]
```

```
  PGA[Payment Gateway Adapter]
```

```
  EH[Emergency Handler]
```

```
end
```

```
subgraph External ["External Interfaces"]
```

```
  PH[PayHere Checkout API]
```

```
  JM[Jitsi Meet SDK]
```

```
  TW[Twilio SMS API]
```

```
  NM[Nodemailer Email Service]
```

```
  GM[Google Maps API]
```

```
  DB[(MongoDB Atlas)]
```

```
end
```

```
%% Frontend Interactions to Backend Services
```

```
AM -->|Authenticates via JWT| AC
```



High-Level System Architecture Specification – SithCare Channeling App

PD -->|Books/Views Appointments| AS

PD -->|Initiates PayHere Sandbox Session| PGA

DP -->|Views Daily Appointments & Activities| AS

CAP -->|Manages Clinic Activities, Appointments & Doctors| AS

CAP -->|Updates Live Queue Placement| QM

SAP -->|Tracks and Manages All Three Portals & Global Metrics| AC

%% Direct Front-End Script Integrations

VC -->|Establishes WebRTC Streaming| JM

PD -->|Renders Interactive Clinic Locations| GM

%% Backend Service to External Systems

PGA -->|Verifies Transactions| PH

EH -->|Dispatches Crisis Alerts| NS

AS -->|Triggers Notification Payload| NS

NS -->|Sends Real-Time Text Notifications| TW

NS -->|Sends Secure System Verification Mail| NM

%% Backend to NoSQL Persistence Layer

AC -->|Performs CRUD on User Collections| DB

AS -->|Persists Schedule & Booking States| DB

QM -->|Mutates Active Queue Positions| DB



5. Deployment Diagram Specifications

The deployment topology utilizes free-tier cloud services optimized for MERN stack applications, adhering to the project's zero-cost infrastructure constraint.

- Client Devices: Web Browsers (Chrome, Firefox, Safari) accessing the SPA via HTTPS (TLS 1.3).
- Frontend Hosting (e.g., Vercel / Netlify): Hosts static React assets and provides a global CDN for fast initial load times.
- Backend Web Server (e.g., Render / Railway): Node.js instance running the Express API, exposed via a reverse proxy handling SSL termination.
- Database Cluster (MongoDB Atlas Free Tier): Cloud-hosted MongoDB cluster handling all data persistence.

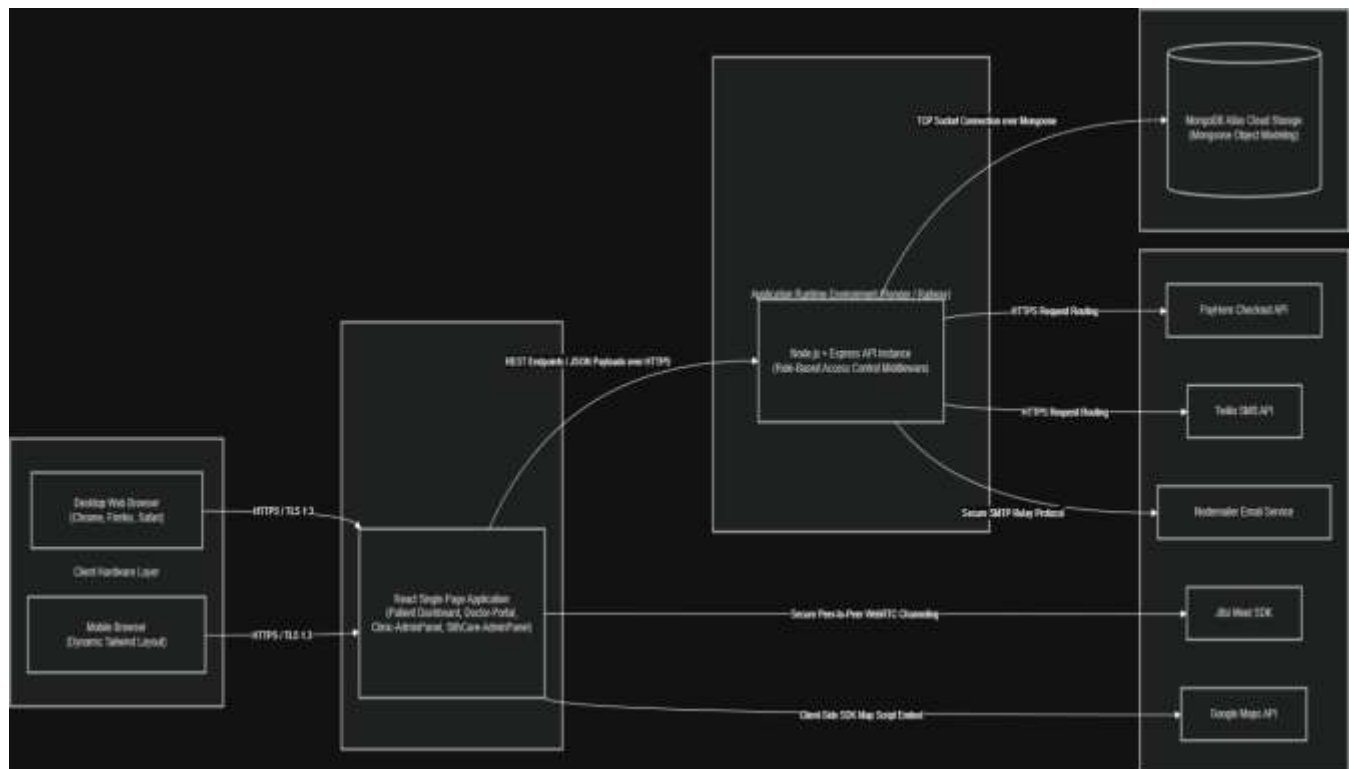


Figure 2 : DeploymentDiagram



High-Level System Architecture Specification – SithCare Channeling App

Mermaid.js Diagram source code

flowchart LR

```
subgraph Clients ["Client Hardware Layer"]
```

```
    Browser["Desktop Web Browser\n(Chrome, Firefox, Safari)"]
```

```
    MobileBrowser["Mobile Browser\n(Dynamic Tailwind Layout)"]
```

```
end
```

```
subgraph CDN ["Static Distribution Layer (Vercel / Netlify)"]
```

```
    SPA["React Single Page Application\n(Patient Dashboard, Doctor-Portal,\nClinic-AdminPanel, SithCare-AdminPanel)"]
```

```
end
```

```
subgraph AppServer ["Application Runtime Environment (Render / Railway)"]
```

```
    NodeAPI["Node.js + Express API Instance\n(Role-Based Access Control Middleware)"]
```

```
end
```

```
subgraph DBServer ["Database Cluster Layer"]
```

```
    MongoCluster[("MongoDB Atlas Cloud Storage\n(Mongoose Object Modeling)")]
```

```
End
```

```
subgraph External ["Third-Party External Interfaces"]
```

```
    PH["PayHere Checkout API"]
```

```
    JM["Jitsi Meet SDK"]
```

```
    TW["Twilio SMS API"]
```

```
    NM["Nodemailer Email Service"]
```

```
    GM["Google Maps API"]
```

```
end
```



High-Level System Architecture Specification – SithCare Channeling App

%% Execution Network Vectors

Browser -->|HTTPS / TLS 1.3| SPA

MobileBrowser -->|HTTPS / TLS 1.3| SPA

SPA -->|REST Endpoints / JSON Payloads over HTTPS| NodeAPI

SPA -->|Secure Peer-to-Peer WebRTC Channeling| JM

SPA -->|Client-Side SDK Map Script Embed| GM

NodeAPI -->|TCP Socket Connection over Mongoose| MongoCluster

NodeAPI -->|HTTPS Request Routing| PH

NodeAPI -->|HTTPS Request Routing| TW

NodeAPI -->|Secure SMTP Relay Protocol| NM